HW2 Instructions - Minitorch Implementation

Homework 2 - Minitorch Implementation

COMPSCI 532

Due 10/5/2026 11:59 PM EST

General Instructions

- Work in groups of two for this assignment

Assignment

In this homework, you will be working to implement elements of MiniTorch. MiniTorch is an implementation of the popular Torch deep learning API. Torch is a popular machine learning library developed by Meta AI. MiniTorch is simplified implementation designed for teaching the code development process for deep learning platforms. Later in the semester will be developing applications using a similar API, PyTorch. The later homework will demonstrate the application of the Torch API, rather than the implementation.

Many parts of this assignment are borrowed from the open-source Mini-Torch example assignment repository linked here.

HW2 - Minitorch Implementation Academic Integrity Policy

MiniTorch is a publicly available open-source project. As such, solutions are available in various places online. If you choose, you may use these solutions as reference while completing the programming sections in this assignment. However, all members of each group will be responsible for understanding both the syntax and underlying concepts. We trust that you will take advantage of outside resources in a responsible manner that supports your learning goals.

The analysis and conclusion report may not use outside resources and should not be discussed outside of each respective assignment group.

If you have any questions or concerns about the academic integrity policy for this assignment, please direct them to the course staff on Piazza.

The above policy is unique to this assignment and does not apply to past or future assignments. See both the syllabus AND the academic integrity policies supplied for each individual assignment, as policies differ for each assignment.

Learning Goals

In this assignment, you will be practicing a variety of systems skills:

- Installing and setting up a programming environment
- Using the git version control software
- Working in groups to develop software and complete written tasks
- Coding within an existing codebase
- Testing and debugging with unit tests
- Implementing various machine learning operations
- Developing higher-order functions using functional Python
- Implementing a tree-structure to support autodifferentiation

Assignment Instructions

Part 1: Minitorch and Coding Environment Setup

This section provides instructions for the initial setup of the coding environment. At the conclusion of this section you will be able to compile and run the Minitorch unit tests required for this assignment. Please note that at the conclusion of Part 1, the unit tests are expected to mostly fail, as important sections of the code will be incomplete. The incomplete sections will be completed in later sections, at which point the unit tests will eventually pass.

Please follow these setup instructions carefully and in order. If you follow these instructions as written, you will end up with a working setup.

Warning: You may wish to develop your own procedure for configuring a development environment. That is fine. However, the course staff have only verified the given procedure. As such, they will not be able to provide extensive support for alternative configurations.

Step: Install VSCode

Install the Visual Studio Code Interactive Development Environment. VSCode is available free of cost for a variety of operating systems at this link.

Step: Download Starter Code and Set Up VSCode IDE

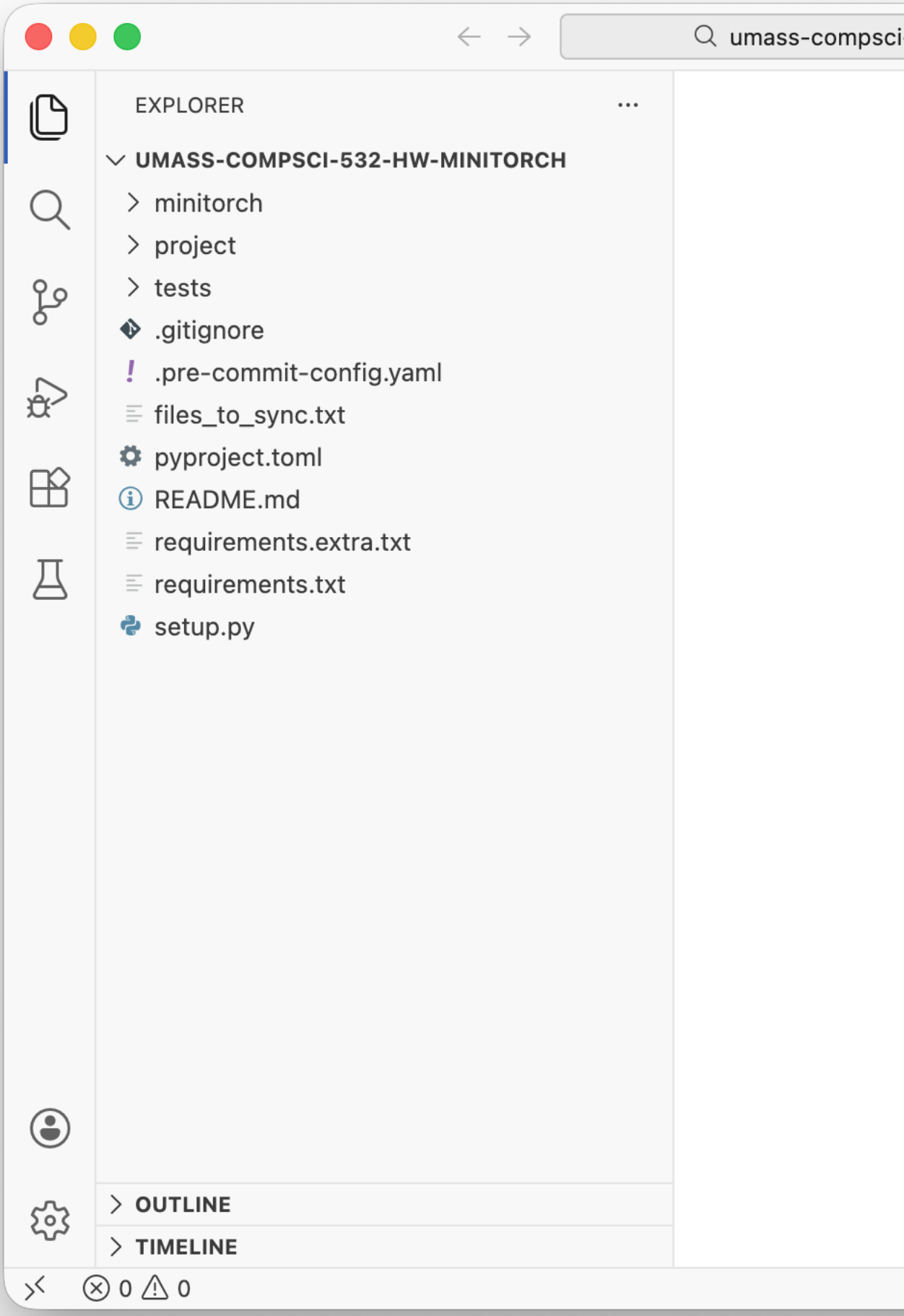
The course staff provide starter for your group to complete later in this assignment. This step will guide you through the process of downloading the starter code and setting it up in the VSCode IDE.

Install Python version 3.14.3 on your system. You can download Python here. Make sure to download the correct OS and version. The specific Python version I used was 3.14.3, later versions will likely work.

Download the starter code from here and unzip the folder on your machine.

Install Visual Studio Code on your computer. You can download it here. Open the unzipped folder: umass-compsci-532-HW-MiniTorch.

You should see something like this.


umass-compsci
EXPLORER
UMASS-COMPSCI-532-HW-MINITORCH
minitorch
project
tests
.gitignore
.pre-commit-config.yaml
files_to_sync.txt
pyproject.toml
README.md
requirements.extra.txt
requirements.txt
setup.py
OUTLINE
TIMELINE
0 0

Set up the python virtual environment. Select View->Command Palette and enter "Python: Create Environment" into the command palette. A pull-down should appear, select Venv, select Python 3.14, and check both requirements.txt and requirements.extra.txt. Press ok. You should see a progress indicator in the bottom-right of the VSCode window. Wait for VSCode to create the environment before moving on. This will take about a couple minutes.

Next confirm your successful installation. Open a new terminal in VSCode: Terminal->New Terminal. The new terminal pane should appear at the bottom of the VSCode window and the command line prompt should begin with "((.venv)" indicating that the correct virtual environment has been loaded. Enter the command "python" in the terminal. You should see python open at the terminal and confirm it has version 3.14. Finally, enter "import minitorch" into the python prompt. If no errors appear, then minitorch is correctly installed.

STEP: Code and Test Task 1

Now begin Task 1. Task 1 is to implement the basic operators that will become part of a deep learning neural network. The built-in operators are specified by programmers utilizing MiniTorch to instantiate a neural network. Task 1 is to implement the operators that are instantiated by these MiniTorch users.

Open the file minitorch/operators.py You will see an example implementation of the mul operator. Complete the remaining operators id, add, neg, ..., inv_back, and relu_back. The required code is marked with TODOs. Insert the code for Task 1 there.

After completing the implementations for each of the required operators, run the required unit tests for task 1 as follows:

```
pytest tests/test_operators.py -m task0_1
```

If you run the task 1 unit tests before completing at least stub implementations for each of the required operators, you will encounter an error indicating that the operator name cannot be imported from minitorch.operators.

Screenshot your terminal to show completion of Task 1 to demonstrate completion of the Task 1 unit tests. Insert that screenshot into the HW2 Report Template described in the Deliverables section of this document.

STEP: Code and Test Task 2

The overarching goal of Task 2 is to implement unit tests for the operators you implemented in Task 1.

Templates for each of the required tests are provided in the file tests/test_operators.py. Insert the code for Task 2 there.

Each required unit test (also known as a property test in this case) is marked with a TODO. Instructions for the type of test to perform are also provided. Your submission must follow these instructions. It may seem tempting to hard code the unit test to always pass, but submissions that implement the required testing strategy will not receive credit.

You may use the provided unit tests for Task 1 as examples for how to write a unit test in the pytest framework. Using the pytest framework is not optional for this assignment.

You may run your Task 2 unit tests using the following command:

```
pytest tests/test_operators.py -m task0_2
```

Your Task 1 code must meet the requirements of the Task 2 unit tests. You may need to modify your Task 1 code. In that case, recheck that the Task 1 unit tests still pass.

Screenshot your terminal to show completion of Task 2 to demonstrate completion of the Task 2 unit tests. Insert that screenshot into the HW2 Report Template described in the Deliverables section of this document.

STEP: Code and Test Task 3

The main goal of Task 3 is to practice functional programming in Python. Functional Programming is a commonly used approach to writing highly parallelizable code. You will implement the following basic functional concepts: map, zipWith, and reduce. Then you will demonstrate their use in combination with the operators already written in Task 1.

The file minitorch/operators.py includes a TODO for Task 0.3 that includes the requirements for both the basic functional concepts and the Task 3 unit tests. Insert the code for Task 3 there.

Here are some notes on functional programming in Python.

https://minitorch.github.io/module0/functional/

You may run your Task 3 unit tests using the following command:

```
pytest tests/test_operators.py -m task0_3
```

Screenshot your terminal to show completion of Task 3 to demonstrate completion of the Task 3 unit tests. Insert that screenshot into the HW2 Report Template described in the Deliverables section of this document.

STEP: Code and Test Task 4

The main goal of Task 4 is to complete the Module class. The modules store learning parameters for a neural network computation tree. The neural network computation tree is implemented as a corresponding tree of Module instantiations. Your specific coding task for Task 4 is completing the Module methods train, eval, named_parameters, and parameters.

Background information about the minitorch Module class can be found here:

https://minitorch.github.io/module0/modules/

The file minitorch/module.py includes TODOs for each coding task in Task 0.4. Complete the code for Task 4 there.

Make sure you can pass the unit tests for Task 4 using the following command:

```
pytest tests/test_module.py -m task0_4
```

PLEASE NOTE: the unit test for task0_4 should pass four tests, but also 1 deselected and 1 xfailed, This is the expected result when the correct solution has been reached. xfail is a decorator used by pytest to mark a unit test that is expected to fail.

Please see this pytest xfail documentation for a more complete explanation of the usage of xfail and its corresponding xpass (where the test was expected to fail, but passes!). Pretty confusing huh!

You may find it helpful to visualize the Module trees using the "Module Sandbox" option of the MiniTorch GUI. The GUI is available with the following command:

```
streamlit run app.py -- 0
```

You may find this task particularly difficult compared to Tasks 1-3. To successfully complete the Module class, you must learn about the Module class and integrate that with tree implementation concepts from your earlier coursework. Look at the completed Module class methods for help in understanding the purpose of the Module instance variables.

Screenshot your terminal to show completion of Task 4 to demonstrate completion of the Task 4 unit tests. Insert that screenshot into the HW2 Report Template described in the Deliverables section of this document.

STEP: Visualization Task 5

The goal of Task 5 is to exercise the code that you implemented in the previous Tasks 1-4.

Visualize the operators you just implemented and tested with the following command:

```
streamlit run project/app.py -- 0
```

Once you have the visualizer open, choose the page "Module 0: Manual." Scroll down to the "Model Parameters" heading. There you can visualize the effect of setting various linear model parameters on learning a dataset.

Below the model parameter sliders is an image of the dataset. The dataset consists of two classes, red X's and the and blue circles.

Your goal for Task 5 is to manually adjust the model parameter sliders to obtain a classification that successfully classifies the Diag dataset with 50 points. You can rely on simple visual confirmation of classification by examining the classification gradient in the image. The colors of the background should roughly correspond to the dataset class colors. Red points should be in mostly red background regions. Blue points should be in mostly blue regions.

Since we have not covered linear models in the class, and that topic is not really a Systems for Data Science concept, you do not have to deeply understand the underlying meaning of the parameter settings. You can just experiment by moving the sliders and observe the changes in the image to see if they improve or worsen the match.

Once you feel you have a good match, make a screenshot of the parameters and resulting classification visualization image. You will insert this screenshot into the HW2 report in the appropriate section of the report template (see deliverables section below). The following image demonstrates a successful manual tuning of the parameters of the Simple dataset and includes both of the required elements: the parameters and the visualization image. Remember that you are being asked to use the Diag dataset.

Model

Parameters

Parameter: linear.weight_0_0

-0.71

-10.00

Parameter: linear.weight_1_0

-0.09

-10.00

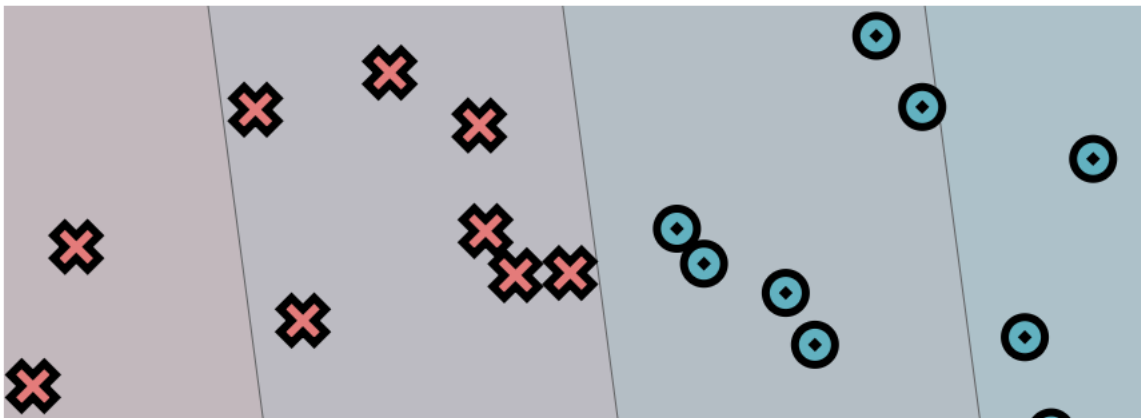
Parameter: linear.bias_0

0.20

-10.00

Show X-Axis Only (For Simple)

Initial setting



To close the visualizer, close the browser tab containing the visualizer, enter the VSCode Terminal window that you launched the visualizer from, and type ctrl-C to quit the streamlit command.

Possibly Useful Documentation

- MiniTorch Project Website Main Page
- MiniTorch Fundamentals Assignment Page that this Assignment closely follows. Note that the instructions on this page have some crucial differences from the instruction on the linked page, so you should only use the MiniTorch web page as an additional resource.
- PyTest usage instructions. This site also has documentation of pytest generally should you be interested in more material.

Deliverables

Your deliverable is a short report to be submitted on Gradescope. Your code will also be submitted and graded.

Groups will submit a single joint submission using the Gradescope group submission tool. Do not submit the assignment individually for each group member.

Your project folder should be zipped and uploaded to the Gradescope Assignment: "HW2 - Project Code Zip Upload." Make sure to include all group members.

Please use the linked Google Doc example document as a template for the report. Your report should be uploaded to the Gradescope assignment: "HW2 - Minitorch Implementation." The graders will appreciate your use of the common template as it will greatly simplifying grading. Using the common template will also benefit you by reducing grading errors.

To use the common template, open the linked Google Doc and click File Menu->Make a Copy. Fill in the required deliverables according to their respective sections. To submit, export as a PDF, click File Menu->Download->PDF.